

Durée : 16h (4h/semaine en 4 semaines) **Atelier de professionnalisation - Semestre 2**

1. Compétences abordées

- B1.2 Recenser et identifier les ressources numériques
 - B1.3 Développer la présence en ligne de l'organisation
 - B2.1 Concevoir et développer une solution applicative
 - B2.2 Assurer la maintenance corrective ou évolutive d'une solution applicative
-

2. Contexte

L'agence de location "AutoRent"

AutoRent est une agence de location de véhicules qui souhaite moderniser la gestion de son parc automobile. Actuellement, les données des véhicules sont reçues quotidiennement du siège sous forme de fichiers CSV bruts, difficiles à exploiter directement.

L'agence a besoin de deux outils complémentaires :

1. Une **application backend (C#)** pour traiter, valider et structurer les données reçues.
 2. Une **interface web (HTML/CSS/PHP)** pour présenter le catalogue des véhicules disponibles aux clients.
-

3. Recherche Préliminaire (Veille Technologique)

Avant de commencer le développement, vous devez acquérir les bases théoriques nécessaires pour cette mission.

Thème : L'Héritage et le Polymorphisme en POO

Vous devez produire une courte **note de synthèse** (1 page max) répondant aux questions suivantes :

1. **L'Héritage :**
 - Qu'est-ce que l'héritage (: en C#) et quel est son but principal (DRY - Don't Repeat Yourself) ?
 - Quelle est la différence entre une classe "Mère" (Base) et une classe "Fille" (Dérivée) ?
 - Peut-on hériter de plusieurs classes en C# ?
2. **Le Polymorphisme :**
 - Que signifient les mots-clés `virtual` et `override` en C# ?
 - Comment faire pour stocker des `Voiture` et des `Moto` dans une seule et même `List<Vehicule>` ?
3. **Application au projet :**
 - Proposez une ébauche de diagramme de classe pour AutoRent montrant comment `Voiture` et `Moto` héritent de `Vehicule` .

4. Cahier des Charges

Le projet doit respecter les contraintes techniques et fonctionnelles suivantes :

A. Gestion des Données (Backend)

- L'application doit être capable de lire un fichier de données brutes (`flotte.csv`) fourni par le siège.
- L'agence loue désormais trois types de véhicules : **Voitures**, **Motos** et **Camping-Cars**.
- Les données doivent être stockées en mémoire sous forme d'objets structurés en utilisant l'**Héritage** (vu en Recherche Préliminaire) :
 - Une classe mère `Vehicule` contenant les propriétés communes (Immatriculation, Marque, Modèle, Prix, État).
 - Des classes filles avec leurs spécificités :
 - `Voiture` : Nombre de places, Climatisation (Oui/Non).
 - `Moto` : Cylindrée (cc), Permis requis (A/A2).
 - `CampingCar` : Longueur (m), Capacité couchage.
- L'application doit garantir l'intégrité des données (ex: pas de prix négatif, format d'immatriculation valide).

B. Traitements Automatisés

- L'application doit permettre de filtrer les véhicules selon leur état (Disponible, En maintenance, Loué) **ET** selon leur type (Voiture, Moto, Camping-Car).
- Elle doit calculer des indicateurs financiers (ex: revenu potentiel total du parc).
- Simulation de devis. L'utilisateur peut sélectionner un véhicule et une durée pour obtenir le coût total (avec application d'une remise de 10% pour les locations > 7 jours).
- Elle doit générer un fichier d'export propre (`disponibles.csv`) contenant uniquement les véhicules prêts à la location, formaté pour être lu par le site web.

C. Interface Client (Frontend)

- Le site web doit afficher dynamiquement la liste des véhicules disponibles à partir du fichier généré par l'application C#.
- L'interface doit permettre de filtrer l'affichage par catégorie (ex: "Voir uniquement les Motos").
- Le client doit pouvoir visualiser clairement les informations spécifiques (ex: la cylindrée pour une moto).

1. Fonctionnement attendu de l'application C#

L'application backend sera une **application Console** interactive destinée au gestionnaire de parc. Voici le scénario d'utilisation type :

1. **Démarrage** : L'application se lance et charge automatiquement les données depuis le fichier source (`flotte.csv`).
2. **Menu Principal** : L'utilisateur accède à un menu textuel proposant plusieurs actions :
 - *Afficher le parc complet* : Liste tous les véhicules (triés par type).

- *Filtrer par type/état* : "Afficher toutes les Motos disponibles".
- *Rechercher un véhicule* : Par immatriculation.
- *Simuler un devis* : Calculer le prix pour X jours.
- *Afficher les statistiques* :
 - Nombre de véhicules par type.
 - Revenu potentiel total.
 - Véhicule le plus cher et le moins cher.
- *Exporter pour le Web* : Génère le fichier `disponibles.csv`.
- *Quitter*.

3. **Interaction** : L'application doit gérer les erreurs de saisie (ex: choix invalide dans le menu) sans planter.

6. Développement de l'Application C# (Partie 2)

Vous devez concevoir et développer l'application en respectant les principes de la **Programmation Orientée Objet**, spécifiquement l'**Héritage** et le **Polymorphisme**.

Travail à faire :

1. Analyse et Modélisation :

- Concevoir l'architecture des classes. Quelle est la classe de base ? Quelles sont les classes dérivées ?
- Définir les propriétés communes et spécifiques.
- Réaliser le **Diagramme de Classes UML** complet mettant en évidence les relations d'héritage.

2. Implémentation :

- Développer la hiérarchie de classes (`Vehicule` , `Voiture` , `Moto` , `CampingCar`).
 - Utiliser le **Polymorphisme** pour la méthode `AfficherDetails()` (chaque type de véhicule s'affiche différemment).
 - Mettre en place la lecture du fichier CSV. Attention : le fichier CSV contiendra une colonne "Type" qui vous permettra de savoir quelle classe instancier (Factory Pattern simplifié).
 - Implémenter la logique du menu et des filtres.
 - Gérer l'export CSV en incluant les nouvelles colonnes spécifiques.
-

7. Développement de l'Interface Web (Partie 3)

L'interface web est la vitrine pour les clients.

Travail à faire :

1. Structure de la page (`index.php`) :

- Concevoir une page qui lit le fichier `disponibles.csv`.
- Utiliser des icônes (FontAwesome ou Emojis) pour distinguer visuellement les types de véhicules (, , ).

2. Design et Ergonomie (CSS) :

- Créer des "Badges" pour afficher les caractéristiques (ex: Badge "Permis A" pour les motos).
- Mettre en page les résultats de manière grille.

3. Recherche et Filtres (Bonus) :

- Ajouter des boutons pour filtrer l'affichage instantanément (Tout, Voitures, Motos, Camping-Cars).

1. Livrables attendus

- **Le Code Source C#** (Projet Visual Studio complet, code commenté).
- **Le Code Source Web** (Fichiers PHP, CSS).
- **Le Diagramme de Classes** (Indispensable pour valider l'héritage).
- **Un Guide Utilisateur** (PDF).

8. Ressources et Aides

- **Format du fichier CSV source (flotte.csv) :**

 [Télécharger le fichier flotte.csv](#)

```
Type;Immatriculation;Marque;Modele;Prix;Km;Etat;Divers1;Divers2
Voiture;AB-123-CD;Peugeot;208;35;12000;Disponible;5;Oui
Moto;EF-456-GH;Yamaha;MT-07;45;5000;En maintenance;689;A
CampingCar;IJ-789-KL;Fiat;Ducato;120;45000;Loué;7.2;4
```

(Note : Les colonnes Divers1 et Divers2 changent de sens selon le type de véhicule. À vous de les mapper correctement dans votre code !)

Note : Ce projet simule une architecture réelle où un "Back-Office" (C#) prépare les données pour un "Front-Office" (Web). Soigner la communication entre les deux (le format du fichier CSV d'échange).